

EDUGENIE – GOOGLE GEMINI POWERED LEARNING ASSISTANT

PROJECT PLAN

1. Project Overview

Project Name: EduGenie – Google Gemini Powered Learning Assistant

Project Type: Generative AI Educational Web Application

Programming Language: Python

Backend: FastAPI

Frontend: HTML, CSS, Jinja2

AI Service: Google Gemini

Optional Local Model: LaMini-Flan-T5-783M

Testing: Pytest

EduGenie is an AI-powered learning assistant that helps students with question answering, concept explanation, quiz generation, text summarization, and personalized learning recommendations.

2. Project Objectives

The main objectives are:

1. Develop an AI-powered educational assistant.
 2. Integrate Google Gemini for Generative AI functionality.
 3. Provide a simple web-based learning interface.
 4. Implement five core learning features.
 5. Develop REST APIs using FastAPI.
 6. Provide automated testing.
 7. Maintain a modular and easily extendable system.
 8. Prepare the application for future educational enhancements.
-

3. Project Scope

The project includes:

- Question and Answer.
- Concept Explanation.
- Quiz Generation.
- Text Summarization.
- Learning Recommendations.
- FastAPI backend.
- HTML/CSS/Jinja2 frontend.
- Gemini API integration.
- Optional local explanation model.
- Automated testing.
- API documentation.
- Health-check functionality.

The current implementation is intentionally lightweight and does not include persistent user accounts, progress databases, or production-scale infrastructure.

4. Development Approach

The project follows a modular development approach:

Requirement Analysis



System Design



Environment Setup



Backend Development



AI Integration



Feature Development



Frontend Development



Integration



Testing



Documentation



Final Demonstration

Each major component is developed and tested before final integration.

5. Technology Selection

Requirement	Technology
Programming	Python 3.10+
Backend	FastAPI
Server	Uvicorn
Frontend	HTML, CSS
Templates	Jinja2
AI	Google Gemini
Optional Local AI LaMini-Flan-T5-783M	
Testing	Pytest
IDE	Visual Studio Code
Version Control	Git/GitHub

The current repository uses a configurable Gemini model, with gemini-2.5-flash as the default.

6. Development Phases

Phase 1 – Requirement Analysis

- Identify the educational problem.
- Define target users.
- Identify required features.
- Define functional and non-functional requirements.
- Determine project scope.

Output: Requirements Document.

Phase 2 – System Design

- Design system architecture.
- Define frontend and backend responsibilities.
- Design API endpoints.
- Design AI processing flow.
- Prepare flowchart and architecture diagram.

Output: Design Document, Flowchart, Architecture Diagram.

Phase 3 – Environment Setup

- Install Python.
 - Create virtual environment.
 - Install dependencies.
 - Configure Gemini API key.
 - Set up Git/GitHub repository.
-

Phase 4 – Backend Development

Develop the FastAPI backend and main API endpoints:

/qa

/explain

/quiz

/summarize

/learn/recommendations

/health

FastAPI also provides interactive documentation through /docs.

Phase 5 – AI Integration

Integrate Google Gemini for:

- Question answering.
- Concept explanation.
- Quiz generation.
- Summarization.
- Learning recommendations.

The optional LaMini-Flan-T5-783M model can be used for local concept explanation.

Phase 6 – Frontend Development

Develop the browser interface using:

- HTML.
- CSS.
- Jinja2.

The interface allows users to select a learning function, enter content, submit a request, and view the generated result.

Phase 7 – Integration

Connect the frontend with the FastAPI backend.

User



Web Interface



FastAPI



AI Processing



Generated Result



Web Interface

All five major learning functions are verified through the complete application workflow.

Phase 8 – Testing

Testing includes:

- Unit testing.
- API testing.
- Functional testing.
- Integration testing.
- Error handling.
- Frontend testing.
- Regression testing.

The project uses Pytest, with Gemini responses mocked so automated tests do not require paid API calls.

Test command:

```
pytest -q
```

7. Resources Required

Hardware

- Laptop or desktop computer.
- Minimum 4 GB RAM recommended.

- Internet connection.
- Modern web browser.

The optional local AI model may require additional memory and processing resources.

Software

- Python 3.10+
- Visual Studio Code
- Git
- FastAPI
- Uvicorn
- Jinja2
- Google GenAI/Gemini SDK
- Pytest

8. Risk Management

Risk	Mitigation
Invalid Gemini API key	Verify environment configuration
Gemini API failure	Implement error handling
Network failure	Test application components independently
Slow local model	Keep local model optional
Invalid user input	Validate requests
API rate limits	Add rate limiting for production
API key exposure	Store secrets in environment variables
Application bugs	Use automated and manual testing

The repository also identifies authentication, rate limiting, logging, HTTPS, request-size limits, and persistent storage as future production improvements.

9. Quality Assurance

Project quality will be maintained through:

- Modular code structure.
 - Input validation.
 - Automated testing.
 - API testing.
 - Integration testing.
 - Manual browser testing.
 - Error handling.
 - Git version control.
 - Documentation review.
-

10. Deployment Plan

For local execution:

uvicorn main:app --reload

Application:

<http://127.0.0.1:8000>

API documentation:

<http://127.0.0.1:8000/docs>

Health check:

<http://127.0.0.1:8000/health>

These execution steps are documented in the current repository.

11. Final Deliverables

The project submission will include:

- Ideation.pdf
- Requirements.pdf
- Flowchart.png
- Architecture.png

- Design.pdf
 - Project_Plan.pdf
 - Testing.pdf
 - Project_Report.pdf
 - Demo_Link.txt
 - Complete source code.
 - Configuration and dependency files.
 - Test files.
-

12. Project Completion Criteria

The project will be considered complete when:

1. All five core features are functional.
 2. FastAPI starts successfully.
 3. The web interface works correctly.
 4. Gemini integration works with a valid API key.
 5. API endpoints return expected responses.
 6. /health works correctly.
 7. /docs is accessible.
 8. Automated tests pass.
 9. API keys and secrets are protected.
 10. All project documentation is completed.
-

13. Conclusion

The EduGenie Project Plan provides a structured approach for developing and delivering an AI-powered educational assistant.

The project follows a modular workflow covering **requirements, design, development, AI integration, frontend development, testing, documentation, and final demonstration.**

The completed system provides:

**Q&A → Concept Explanation → Quiz Generation → Summarization → Learning
Recommendations**

The modular architecture also allows future extensions such as authentication, progress tracking, adaptive learning, analytics, and multimodal AI.